Backtracking Search:- (1) Given a initial partial solution.

State:- Given Variables with assigned values

Successor Function:- Take another unassigned variable and assign it a value.

Goal Test:- All variables are assigned to values, and all constraints satisfied.

Note: All goal nodes are at the same level.

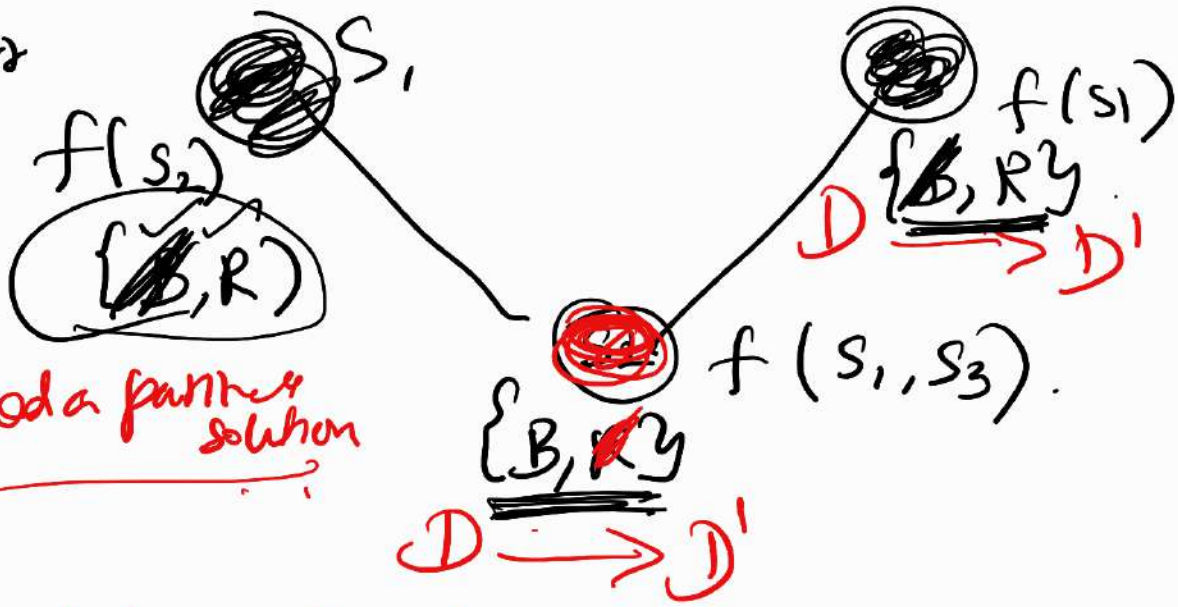
How to improve backtracking:-

(a) Which variables should be assigned next?

(b) In what order, should domain values to be tried?

Empirical Study → * MRV - Minimum Remaining Value
* LCV.

Map-coloring




Assigned a partial solution

## Variable ordering :-

↳ Search problem

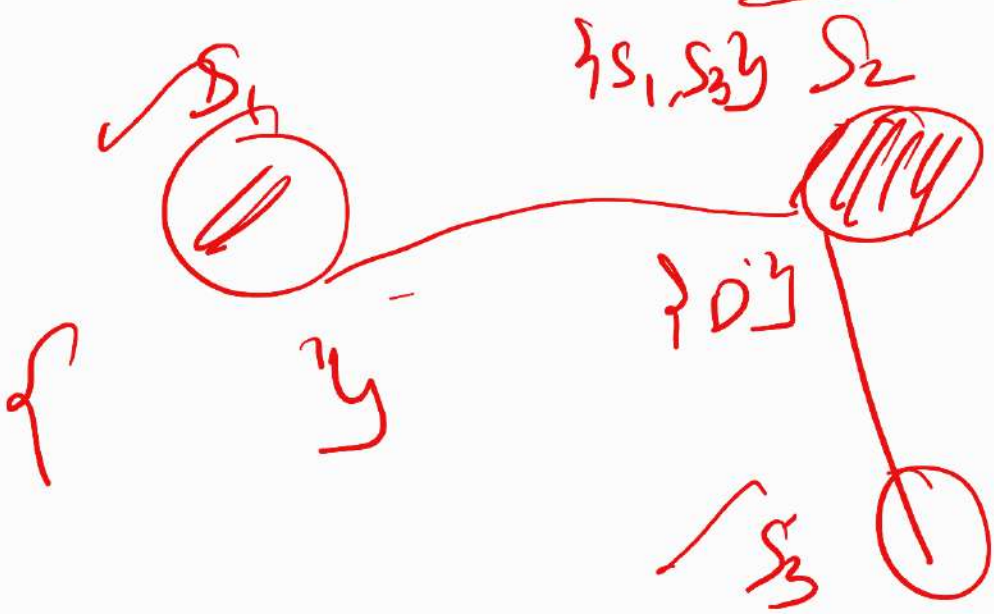
↳ Fine-tune the process of backtracking.

## LCV :-

→ Given the choice of variable, choose the least constraining value.

→ The one that rules out the fewest values in the remaining values.

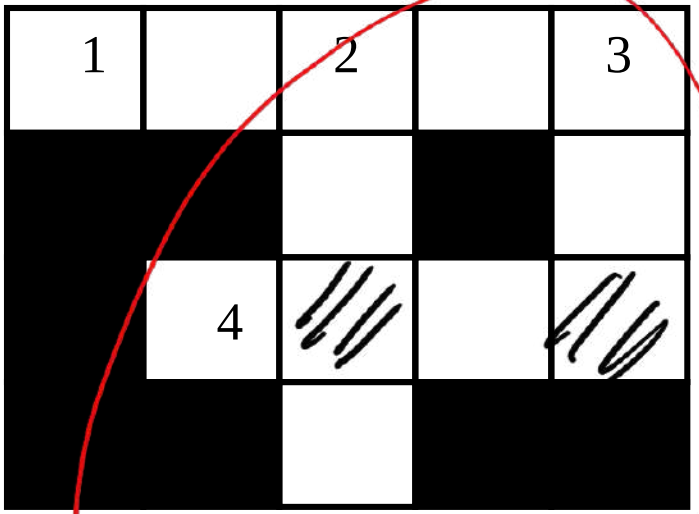
MCV.



# CSP Solution Overview

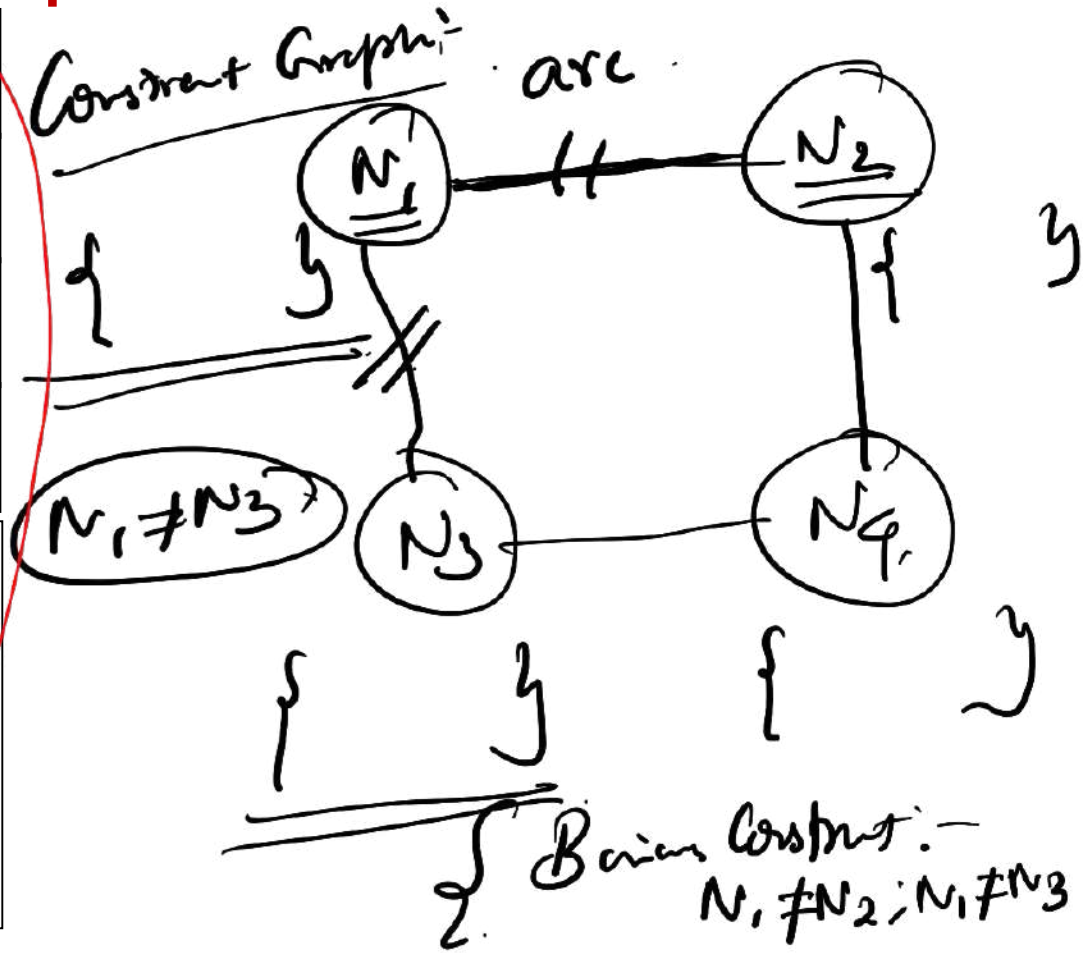
- CSP Graph Creation:
  - Create a Node for Every Variable. All possible Domain Values are initially Assigned to the Variable
  - Draw edges between Nodes if there is a Binary Constraint. Otherwise Draw a hyper-edge between nodes with constraints involving more than two variables
- Constraint Propagation:
  - Reduce the Valid Domains of Each Variable by Applying Node Consistency, Arc / Edge Consistency, K-Consistency, till no further reduction is possible. If a solution is found or the problem found to have no consistent solution, then terminate
- Search for Solution:
  - Apply Search Algorithms to Find Solutions
  - There are interesting properties of CSP graphs which lead of efficient algorithms in some cases: Trees, Perfect Graphs, Interval Graphs, etc
  - Issues for Search: Backtracking Scheme, Ordering of Children, Forward Checking (Look-Ahead) using Dynamic Constraint Propagation
  - Solving by Converting to Satisfiability (SAT) problems

# CSP Graph for Crossword




**Word List:**

astar, happy, hello,
hoses, live, load, loom,
peal, peel, save, talk,
ant, oak, old

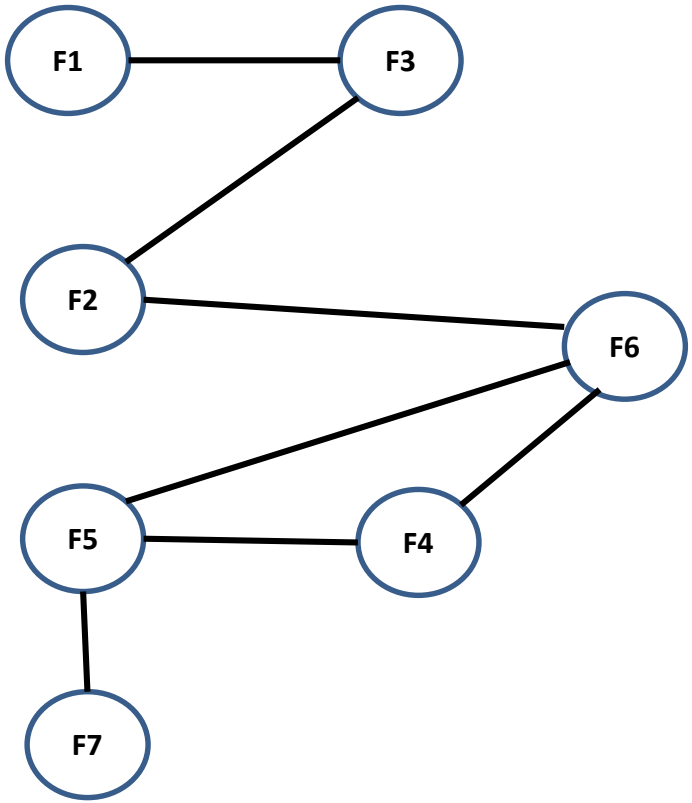


# CSP Graph for Airline Gate Scheduling

| Flight No | Dep Time | G Start | G End |
|---|---|---|---|
| F1 | 7:00 | 6:15 | 7:15 |
| F2 | 8:30 | 7:45 | 8:45 |
| F3 | 7:45 | 7:00 | 8:00 |
| F4 | 9:45 | 9:00 | 10:00 |
| F5 | 10:00 | 9:15 | 10:15 |
| F6 | 9:00 | 8:15 | 9:15 |
| F7 | 11:00 | 10:15 | 11:15 |

# CSP Graph for Airline Gate Scheduling

| Flight No | Dep Time | G Start | G End |
|---|---|---|---|
| F1 | 7:00 | 6:15 | 7:15 |
| F2 | 8:30 | 7:45 | 8:45 |
| F3 | 7:45 | 7:00 | 8:00 |
| F4 | 9:45 | 9:00 | 10:00 |
| F5 | 10:00 | 9:15 | 10:15 |
| F6 | 9:00 | 8:15 | 9:15 |
| F7 | 11:00 | 10:15 | 11:15 |



$\langle x_j, x_i \rangle$ $x_n$

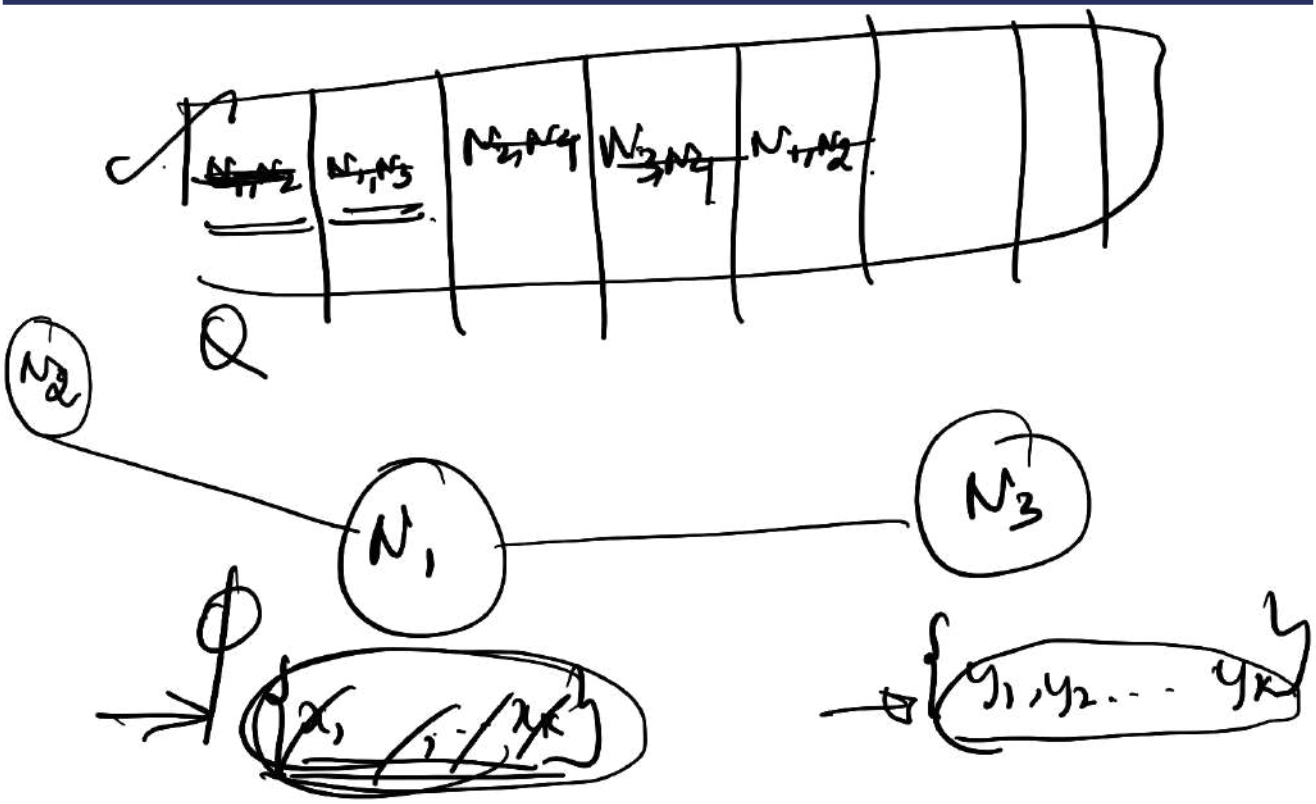
$x_i$ $\{y_1, y_2 \dots y_k\}$

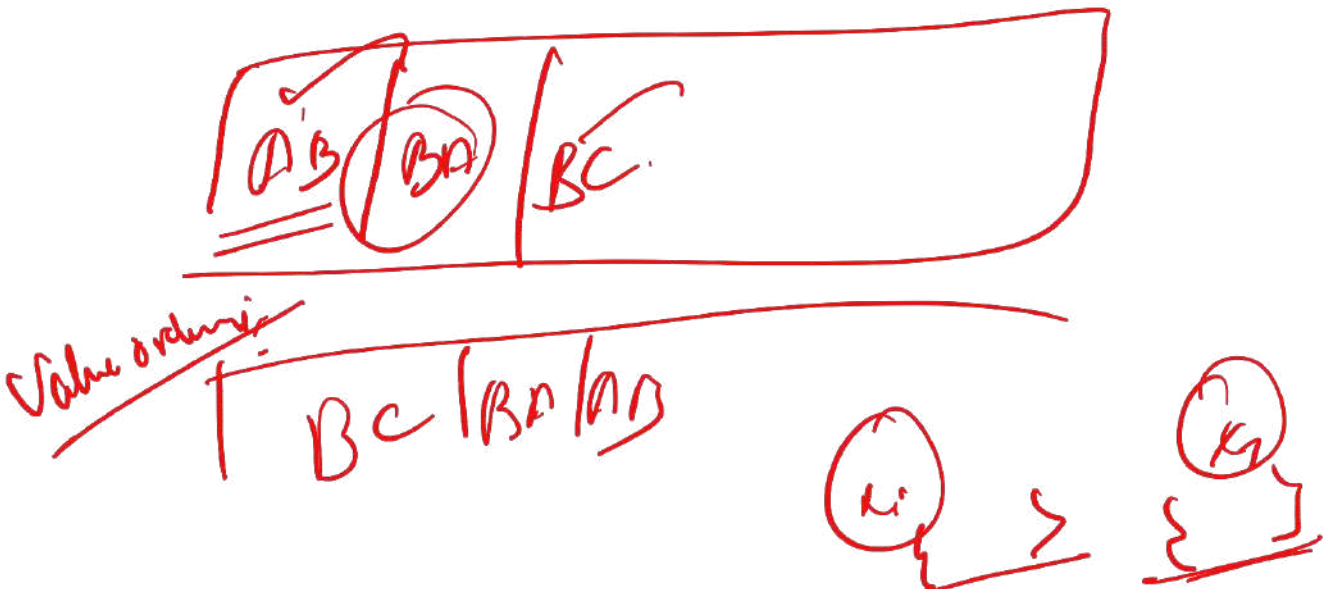
$X_i$ is consistent with $X_j$ iff

# Arc Consistency Algorithm

A variable $X_i$ is **arc consistent** with respect to $X_j$ if for each $x_i \in \text{Domain}_i$, there exists $x_j \in \text{Domain}_j$ such that $f(\{X_i : x_i, X_j : x_j\}) \neq 0$ for all factors $f$ whose scope contains $X_i$ and $X_j$.

$\text{EnforceArcConsistency}(X_i, X_j)$: Remove values from $\text{Domain}_i$ to make $X_i$ arc consistent with respect to $X_j$.



# Constraint Propagation Steps

- Constraints
  - Unary Constraints or Node Constraints
  - Binary Constraints or Edges between CSP Nodes
  - Higher order or Hyper-Edges between CSP Nodes
- Node Consistency
  - For every Variable V_i, remove all elements of D_i that do not satisfy the Unary Constraints for the Variable
  - First Step is to reduce the domains using Node Consistency
- Arc Consistency
  - For every element x_ij of D_i, for every edge from V_i to V_j, remove x_ij if it has no consistent value(s) in other domains satisfying the Constraints
  - Continue to iterate using Arc Consistency till no further reduction happens.
- K-Consistency or Path Consistency
  - For every element y_ij of D_i, choose a Path of length L with L variables, use a consistency checking method similar to above to reduce domains if possible

# AC-3 algorithm:-

$x_i$ $A = \{2, 3\}$

$x_j$ $B = \{1, 2, \}$

$C = \{1, 2\}$

Constraints:-

$A > B$

$B = C$

| Algorithm | Arcs |
|---|---|
| AB | AB $(A > B)$ |
| BA | BA $(B < A)$ |
| BC | BC $(B = C)$ |
| CB | CB $(C = B)$ |
| ~~AB $(A>B)$~~ | |
| ~~BC $(B=C)$~~ | |

$A = \{2, 3\}$

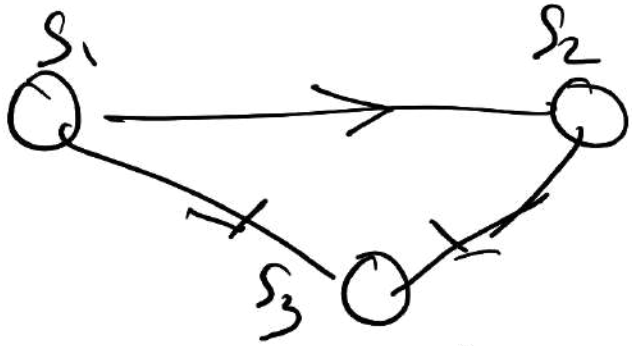
$B = \{1, 2\}$

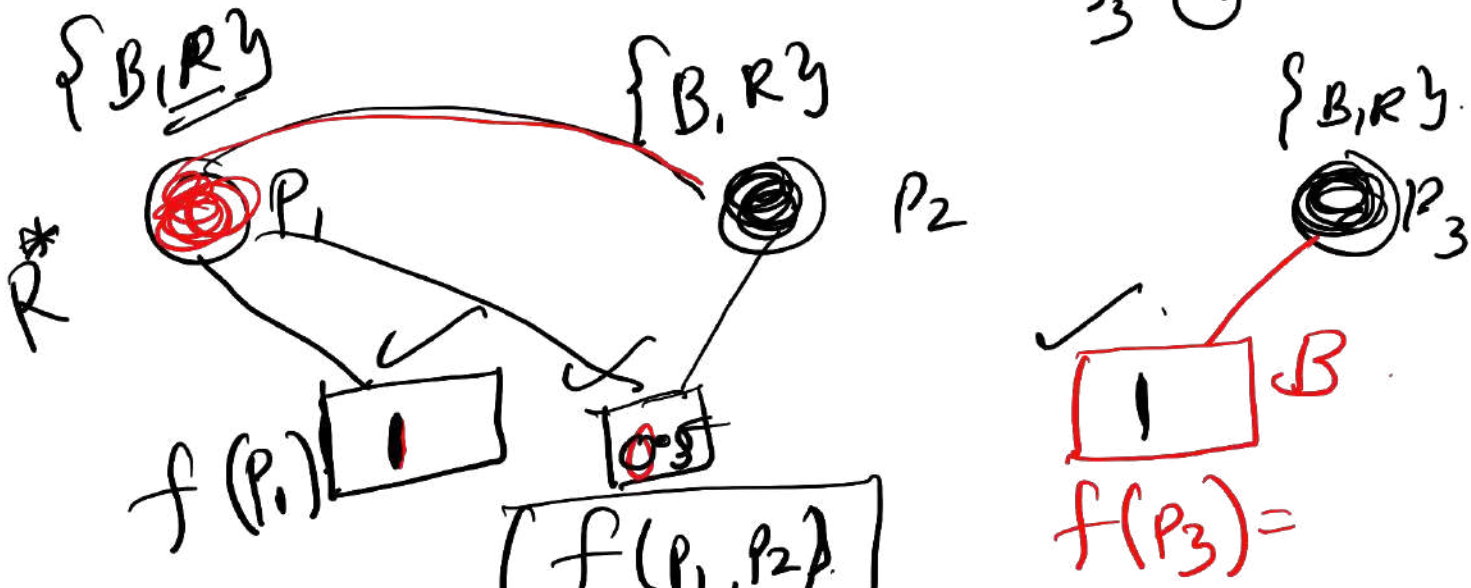
$C = \{1, 2\}$

Empty

| ~~AB~~ | BA | BC | CB | AB | 1 |
|---|---|---|---|---|---|

# Factor-Graph:-




$$f(x) = \arg\max_{x} f_i(x_i)$$

# Arc Consistency Algorithm AC-3

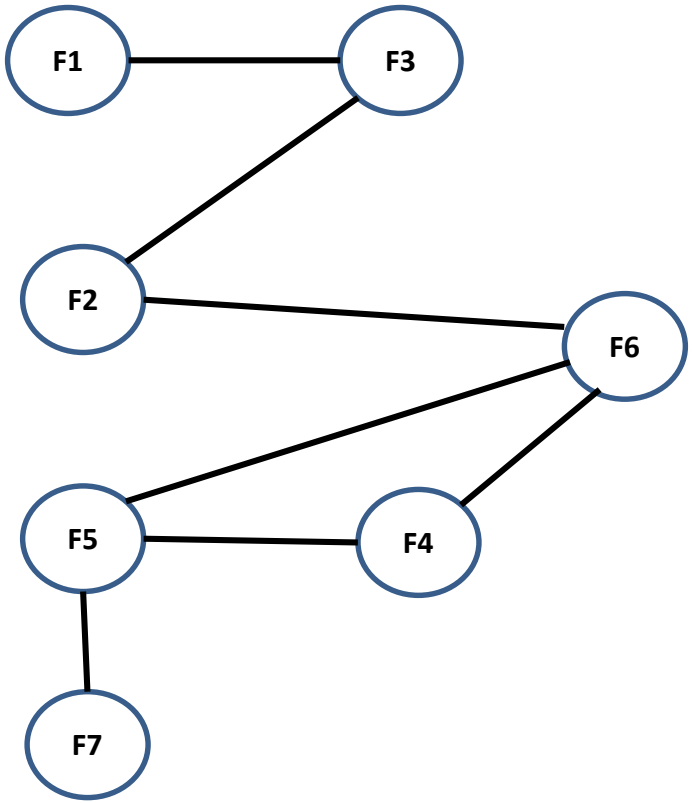
```
AC-3(csp) // inputs - CSP with variables, domains, constraints
    queue ← local variable initialized to all arcs in csp
    while queue is not empty do
        (Xi, Xj) ← pop(queue)
        if Revise(csp, Xi, Xj) then
            if size of Di = 0 then return false
            for each Xk in Xi.neighbors - {Xj} do
                add (Xk, Xi) to queue
    return true
```

```
Revise(csp, Xi, Xj)
    revised ← false
    for each x in Di do
        if no value y in Dj allows (x, y) to satisfy constraint between Xi and Xj then
            delete x from Di
            revised ← true
    return revised
```

Time complexity: $ed^3$

# Consistency for Airline Gate Scheduling

| Flight No | Dep Time | G Start | G End |
|---|---|---|---|
| F1 | 7:00 | 6:15 | 7:15 |
| F2 | 8:30 | 7:45 | 8:45 |
| F3 | 7:45 | 7:00 | 8:00 |
| F4 | 9:45 | 9:00 | 10:00 |
| F5 | 10:00 | 9:15 | 10:15 |
| F6 | 9:00 | 8:15 | 9:15 |
| F7 | 11:00 | 10:15 | 11:15 |



# Backtracking Algorithm for CSP

CSP-BACKTRACKING({})

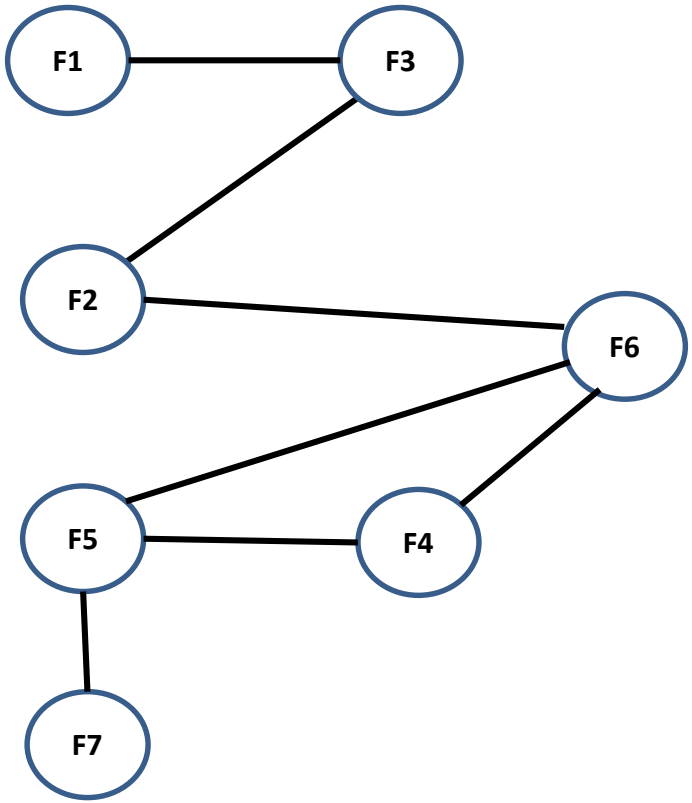
CSP-BACKTRACKING(a)

- If a is complete then return a
- X ← select unassigned variable
- D ← select an ordering for the domain of X
- For each value v in D do
  - If v is consistent with a then
    - Add (X= v) to a
    - result ← CSP-BACKTRACKING(a)
    - If result ≠ *failure* then return result
- Return *failure*

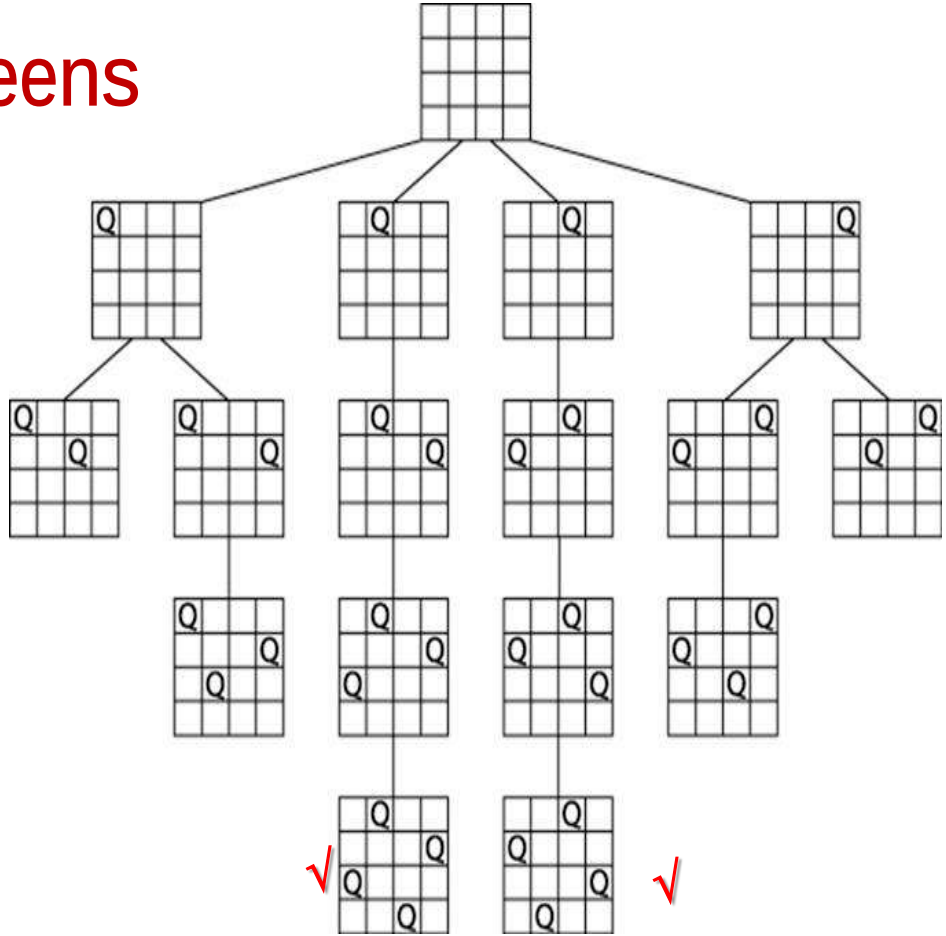
partial assignment of variables

# Backtracking for Airline Gate Scheduling

| Flight No | Dep Time | G Start | G End |
|---|---|---|---|
| F1 | 7:00 | 6:15 | 7:15 |
| F2 | 8:30 | 7:45 | 8:45 |
| F3 | 7:45 | 7:00 | 8:00 |
| F4 | 9:45 | 9:00 | 10:00 |
| F5 | 10:00 | 9:15 | 10:15 |
| F6 | 9:00 | 8:15 | 9:15 |
| F7 | 11:00 | 10:15 | 11:15 |



# Search 4-Queens



```
function ForwardChecking(csp) //returns a new domain for
     each var
    for each variable X in csp do
        for each unassigned variable Y connected to X do
            for each value d in Domain(Y)
                if d is inconsistent with Value(X)
                    Domain(Y)={Domain(Y)-d}
    return csp //whith modified domains
```

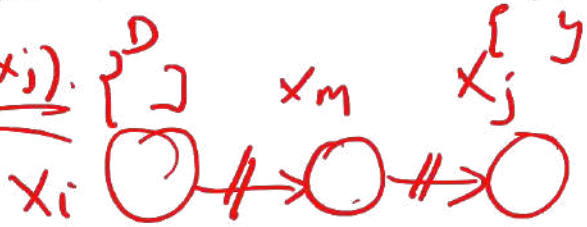
# Path Consistency.

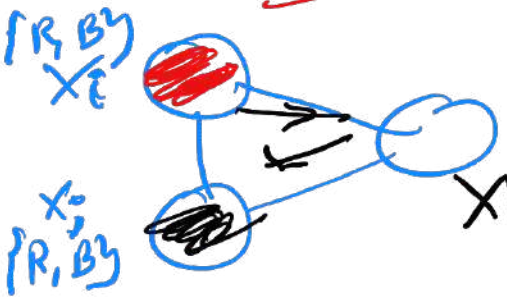
$\boxed{\{X_i, X_j\}}$ is satisfiable (Arc consistent)

$x \in D(X_i) \Rightarrow x \in D(X_j)$,

$\boxed{\exists X_m}$, which also satisfies all the constraints with the assignment of domain values for $(X_i, X_m)$ & $(X_m, X_j)$.

$\{X_i, X_m, X_j\}$




$X_m$ {R, B} $\{X_i, X_m, X_j\}$

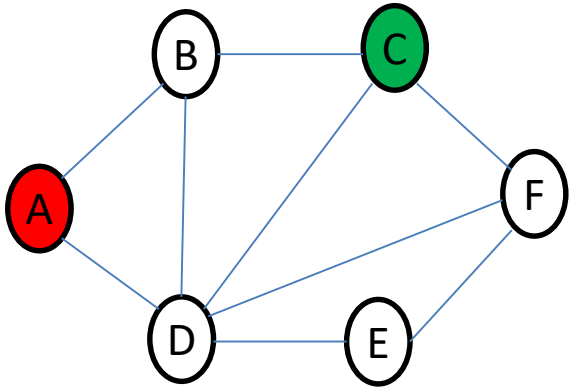
# Strategies for CSP Search Algorithms

- Initial Constraint Propagation → K-arc consistency, AC-3 algorithm.
- Backtracking Search
  - Variable Ordering
    - Most Constrained Variable / Minimum Remaining Values MRV
    - Most Constraining Variable
  - Value Ordering
    - Least Constraining Value leaving maximum flexibility
  - Dynamic Constraint Propagation Through Forward Checking
    - Preventing useless Search ahead
  - Dependency Directed Backtracking
- SAT Formulations and Solvers
- Optimization
  - Branch-and-Bound
  - SMT Solvers, Constraint Programming
- Learning, Memoizing, etc
- CSP Problems are NP-Hard in General → {X, D, C}

① Beam-Search
② Local-Search

# Forward Checking: 3 Colouring Problem

- Forward checking propagates information from assigned to unassigned variables




| A | B | C | D | E | F |
|---|---|---|---|---|---|
| {R, G, B} | {R, G, B} | {R, G, B} | {R, G, B} | {R, G, B} | {R, G, B} |
| | {G, B} | {R, G, B} | {G, B} | {R, G, B} | {R, G, B} |
| | {B} | | {B} | {R, G, B} | {R, B} |

X

- B and D cannot both be blue!
- Why did we not detect this?
- Forward checking detects some inconsistencies, not all
- *Constraint propagation:* reason from constraint to constraint